

Ejercicios combinados

Lista de Ejercicios Combinados

Bloque 1: Fundamentos y Clásicos

Estos son ejercicios esenciales para practicar los conceptos básicos.

1. Saludo Personalizado (Clásico)

Escribe un script que almacene tu nombre en una variable (ej: `nombre="TuNombre"`) e imprima `Hola, TuNombre`.

2. Saludo Interactivo (Clásico)

Escribe un script que pida al usuario su nombre usando `read` y luego imprima `Hola, [nombre_usuario]`.

3. Saludo con Argumentos (Clásico)

Escribe un script que reciba un argumento desde la línea de comandos (ej: `./saludo.sh Ana`) e imprima `Hola, Ana`.

4. Comprobación Numérica (Clásico)

Escribe un script que pida al usuario un número. Usa una sentencia `if` para imprimir "El número es positivo" si es mayor que 0, o "El número es 0 o negativo" si no lo es.

5. Conteo Básico (Clásico)

Escribe un script que use un bucle `for` para imprimir los números del 1 al 10.

Bloque 2: Bucles y Condicionales

Estos ejercicios combinan bucles y lógica condicional.

6. Números Pares e Impares

Escribe un script que recorra los números del 1 al 20. Debe imprimir "El número [X] es par" o "El número [X] es impar" para cada número.

7. Contador con `while` (Clásico)

Escribe un script que use un bucle `while` para contar hacia atrás, desde 10 hasta 1.

8. Contraseña Secreta

Escribe un script que pida al usuario una contraseña. Debe usar un bucle `while` para seguir preguntando hasta que el usuario escriba la palabra "secreto".

9. Recorrer un Array

Define un array que contenga tres nombres de frutas (ej: `frutas=( "Manzana" "Banana" "Fresa" )`).

Usa un bucle `for` para imprimir cada fruta del array en una línea diferente.

10. Comprobación de Archivo (Clásico)

Escribe un script que reciba un nombre de archivo como argumento (`$1`). El script debe comprobar si el archivo existe e imprimir un mensaje (ej: "El archivo [nombre] existe" o "El archivo [nombre] no existe").

Bloque 3: Introducción a Funciones

Aquí empezarás a organizar tu código en funciones reutilizables.

11. Tu Primera Función

Escribe un script que defina una función simple llamada `hola`. Esta función debe imprimir "¡Hola desde mi función!". Llama a la función al final del script.

12. Función con Argumentos

Crea una función llamada `saludar` que acepte un argumento (un nombre). La función debe imprimir `Bienvenido, [nombre]`. Llama a la función pasándole un nombre.

13. Refactorización (Funciones)

Toma el ejercicio 4 (Comprobación Numérica) y conviértelo en una función llamada `comprobar_numero` que reciba el número como argumento.

14. Menú Básico con `case`

Escribe un script que muestre al usuario dos opciones: "1) Mostrar la fecha actual" y "2) Mostrar quién está conectado". Pide al usuario que elija (1 o 2). Usa una sentencia `case` para ejecutar el comando `date` o el comando `who` según la elección.

15. Comprobación de Usuario (Avanzado)

Escribe un script que compruebe si el usuario que lo ejecuta es `root`. Si el `EUID` es 0, debe imprimir "Advertencia: Estás ejecutando como root". En caso contrario, debe imprimir "Ejecutando como usuario normal".

Bloque 4: Scripts Avanzados

Estos ejercicios combinan múltiples conceptos para crear herramientas más útiles.

16. Script de Información del Sistema

Crea un script que tenga tres funciones: `mostrar_memoria` (que ejecuta `free -h`), `mostrar_disco` (que ejecuta `df -h`) y `mostrar_uptime` (que ejecuta `uptime`). Llama a las tres funciones una después de la otra.

17. Menú con Funciones (Refactorización)

Toma el ejercicio 14 (Menú Básico) y convierte cada opción (mostrar fecha, mostrar usuario) en su propia función.

18. Gestión de Argumentos

Escribe un script que espere exactamente un argumento.

- Si no recibe ningún argumento (`$#` no es 1), debe imprimir "Error: Debes proporcionar un nombre de archivo."
- Si recibe un argumento, debe imprimir "Procesando el archivo: \$1".

19. Creación de Directorios

Escribe un script que pida al usuario el nombre de un proyecto. Luego, debe crear un directorio con ese nombre y, dentro de él, crear tres subdirectorios: `src`, `docs` y `tests`.

20. Script de Búsqueda (Avanzado)

Escribe un script que reciba una extensión de archivo como argumento (ej: `.log` o `.txt`). El script debe buscar todos los archivos con esa extensión en el directorio actual y moverlos a un directorio llamado `backup`. Si el directorio `backup` no existe, el script debe crearlo primero.